EN.553.402 / 553.602 · DATA MINING · JOHNS HOPKINS

Predicting Baltimore 311 Resolution Time

Mid-Semester Update

TEAM CHINGU

Manikrishneshwar Sasidhar · Yeo Eun Yoon

November 2026

THE QUESTION AND THE ROAD MAP

Can we predict how long a 311 request takes — and does it work everywhere?

What we are asking

Using only what is known **when a request is filed** — request type, timing, location, responsible agency — can we predict resolution time, which factors matter most, and does accuracy differ across neighbourhoods and request types?

Scope. All agencies, excluding 311 call-centre information requests and city-generated work. Baltimore City Open Data, 2024–2025.

Four models, each on the same rows, the same split and the same metrics.

WHAT THIS UPDATE COVERS

3–4 Data & preprocessing

5–11 Work completed

6–11 Methods implemented

12–16 Preliminary results

17–18 Problems encountered

19 Unsuccessful approaches

20 Changes to the plan

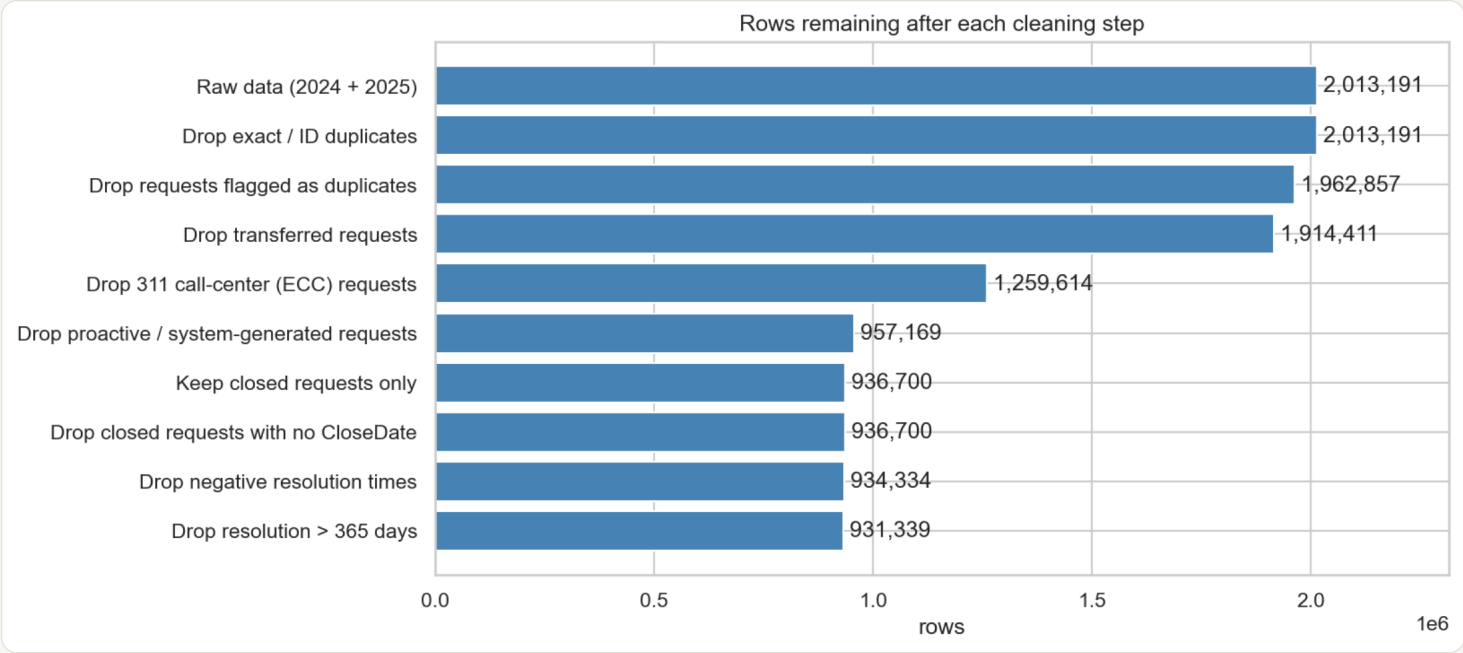
21 Plan to complete

DATA ACQUISITION AND PREPROCESSING

Over half the raw file is not service work

2.01M raw rows

931K modelled



Two cuts do almost all the work: **311 call-centre information requests (−654,797)**, which are questions answered on the phone, and **city-generated proactive jobs (−302,445)** such as scheduled rat baiting. Both close within seconds of being logged, so their “resolution time” is not service work.

METHODS IMPLEMENTED

Training and test split

Train
2024
489,223 requests

Test, scored once
2025
434,299 requests

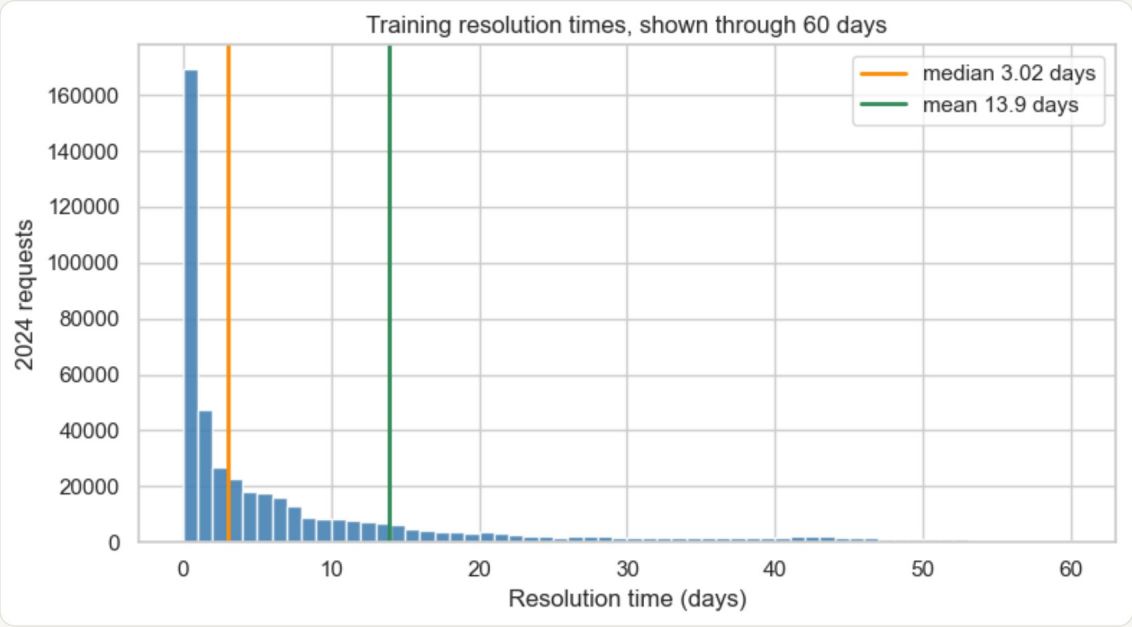
The target is days from the open date to the close date. The regression fits log(1 + days) and converts predictions back to days.

Deadlines outside 0 to 120 days are dropped in both, so both scores use the same **923,522 requests**.

Compared on MAE, RMSE, and R^2. The 2025 year is scored once and is **not used to choose anything**.

MODEL 1 OF 4 · MEDIAN BASELINE

Distribution of resolution time

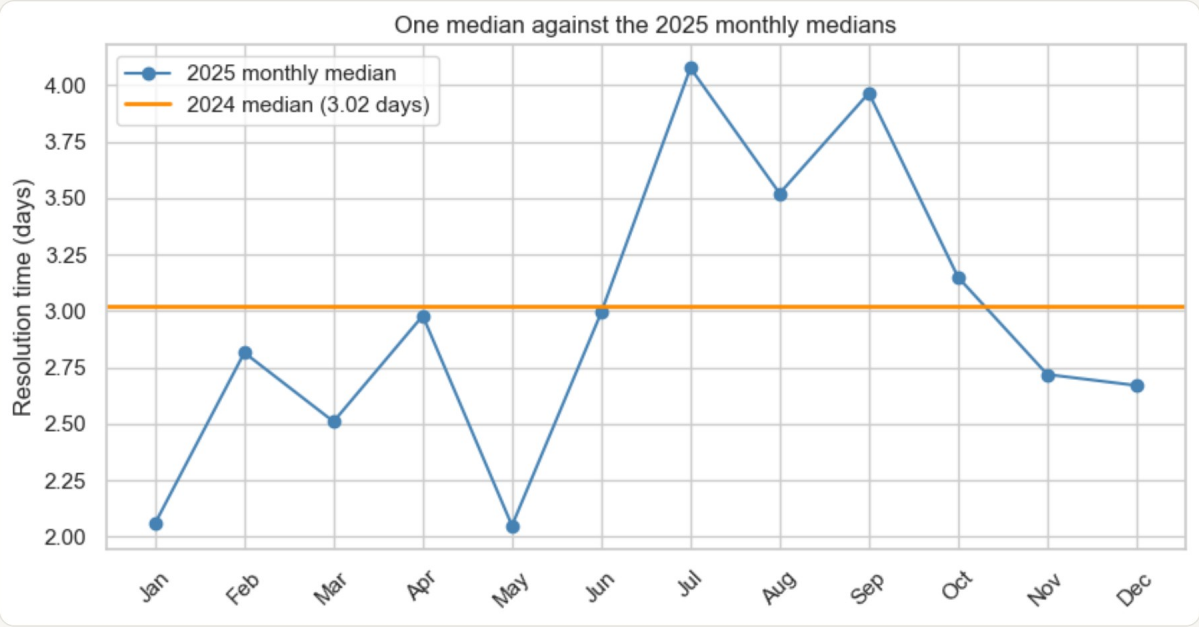


Histogram of 2024 resolution time through 60 days, with the median and the mean marked.

Most requests close in a few days, so the median is 3.02 days and the mean is 13.9 days.

MODEL 1 OF 4 · MEDIAN BASELINE

Monthly median resolution time, 2025

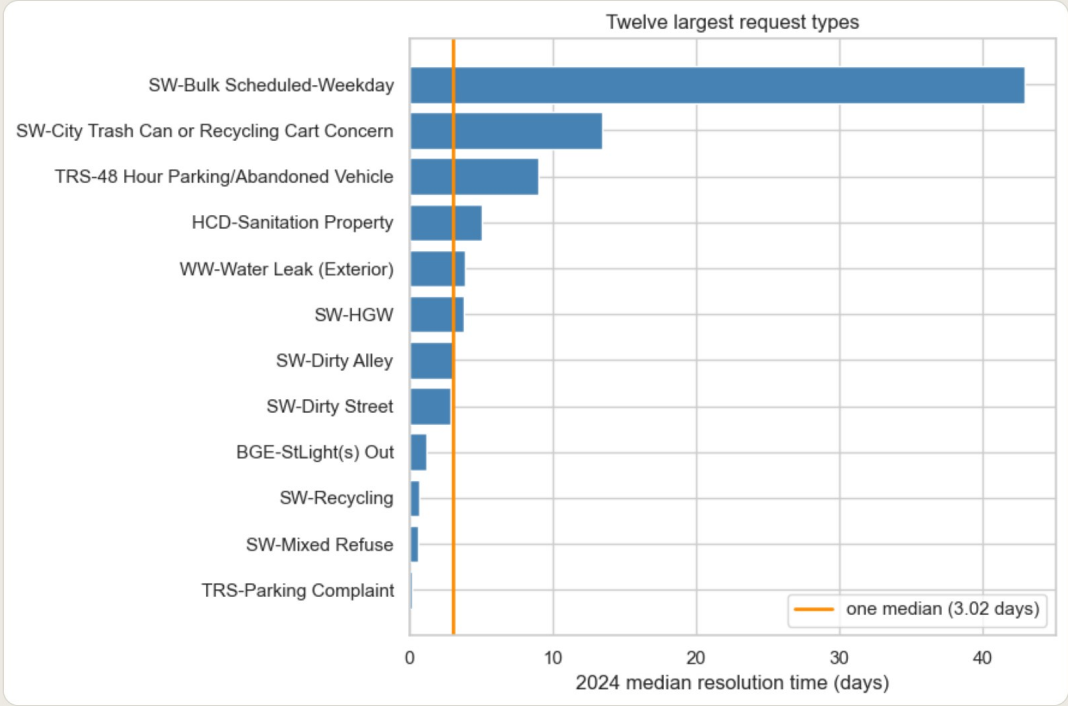


Each point is the median resolution time for requests filed in that month of 2025.

Monthly medians stay near the 2024 median of 3.02 days, so the center of the year barely moves.

MODEL 1 OF 4 · MEDIAN BASELINE

Median resolution time by request type

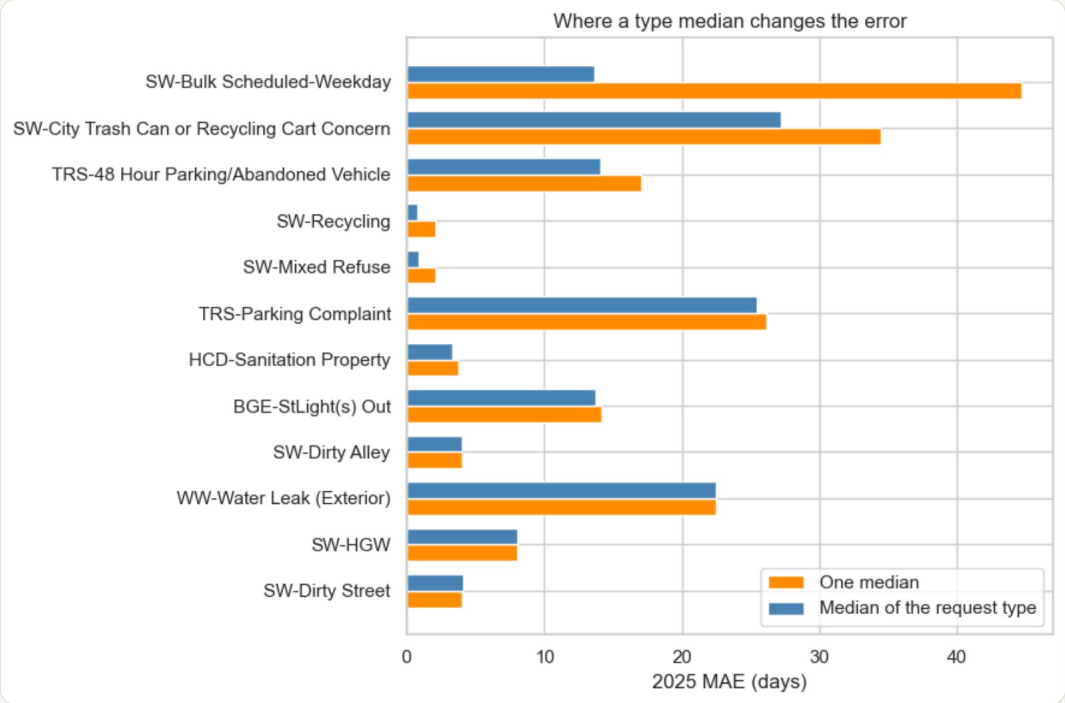


Each bar is the 2024 median for one of the twelve largest request types.

The medians run from under a day for parking to about 43 days for weekday bulk collection.

MODEL 1 OF 4 · MEDIAN BASELINE

Test error by request type



Orange is the 2025 error from guessing 3.02 days; blue is the error from that type's own 2024 median.

The type median cuts error most for the slow types, especially weekday bulk collection.

MODEL 1 OF 4 · RESULT PRELIMINARY RESULTS

Baseline results, 2025 test

Rule	MAE (days)	RMSE	R^2 (days)	R^2 (log)
2024 mean for every request	18.30	36.86	−0.002	−0.563
2024 median for every request	14.71	38.88	−0.115	−0.035
2024 median of each request type	11.92	35.96	0.047	0.264

2025 scores for one mean, one median, and a median for each request type.

The type median is the baseline to beat: typical error falls from 14.7 days to 11.9.

MODEL 2 OF 4 · LINEAR REGRESSION

Fitted equation

$$X = \log(1 + \text{days}) = 1.557 + 0.0287 \times \text{deadline} + \text{type add-on} + \text{month add-on} + \text{weekday add-on} + \text{hour add-on}$$

Predicted days = exp(X) − 1.

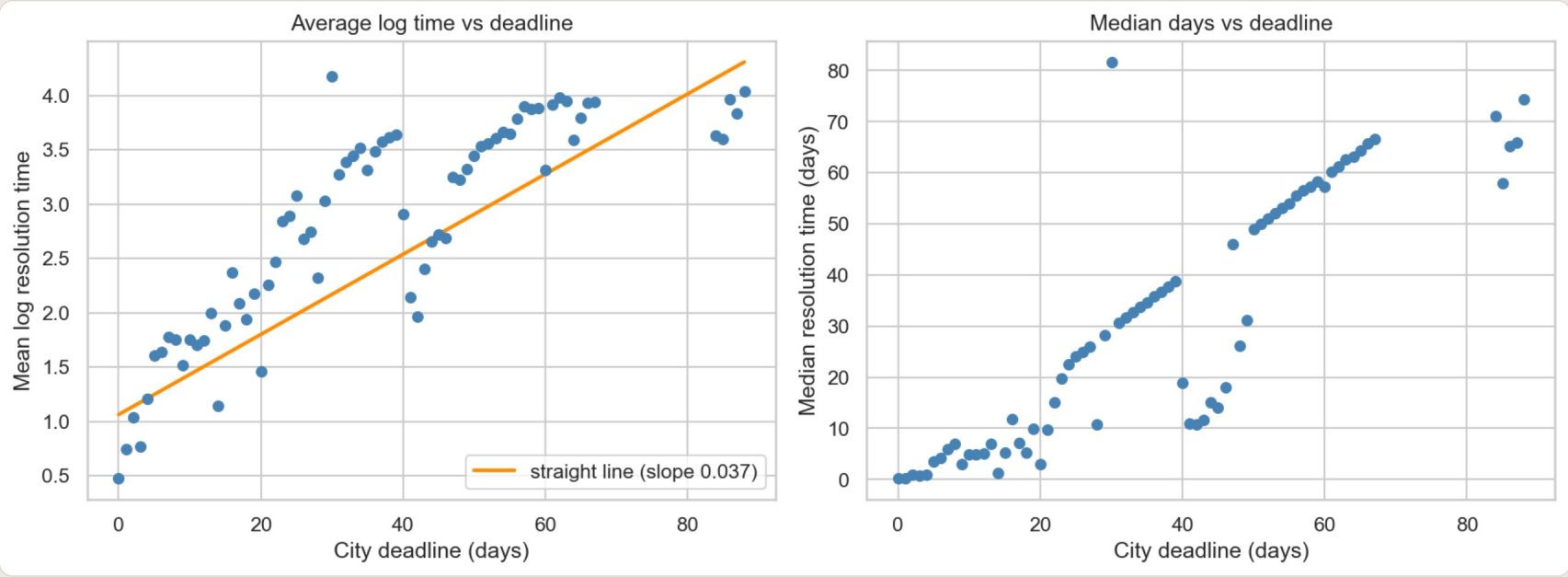
Same 10-day deadline, January, 10:00	Plug into the equation	Predicted days
Sanitation property, Monday (all add-ons 0)	1.557 + 0.0287 × 10	5.3
Same request, filed on Friday	1.844 + 0.10	6.0
Pothole pickup instead of sanitation property	1.844 − 1.62	0.26

Each row changes one term, holding the 10-day deadline, January, and 10:00 fixed.

Switching to a pothole changes the prediction by days; filing on Friday changes it by less than a day.

MODEL 2 OF 4 · LINEAR REGRESSION

Resolution time by city deadline

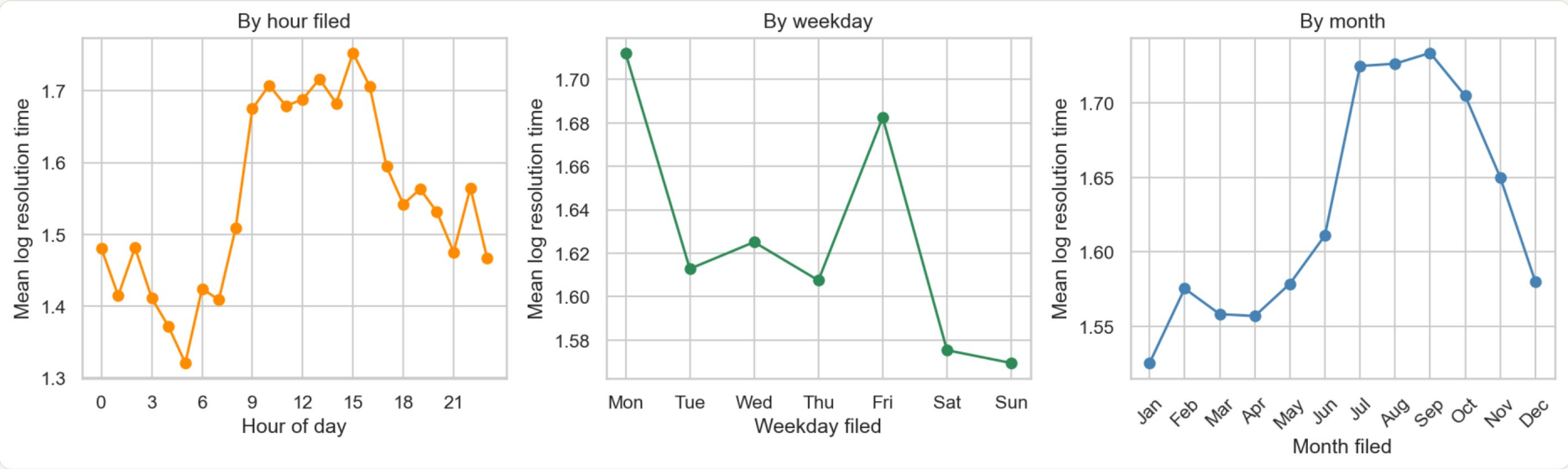


Each dot groups requests with the same deadline: left is average log(1 + days), right is median days.

Longer deadlines go with longer closures, and many deadlines sit off one straight line.

MODEL 2 OF 4 · LINEAR REGRESSION

Resolution time by hour, weekday, and month

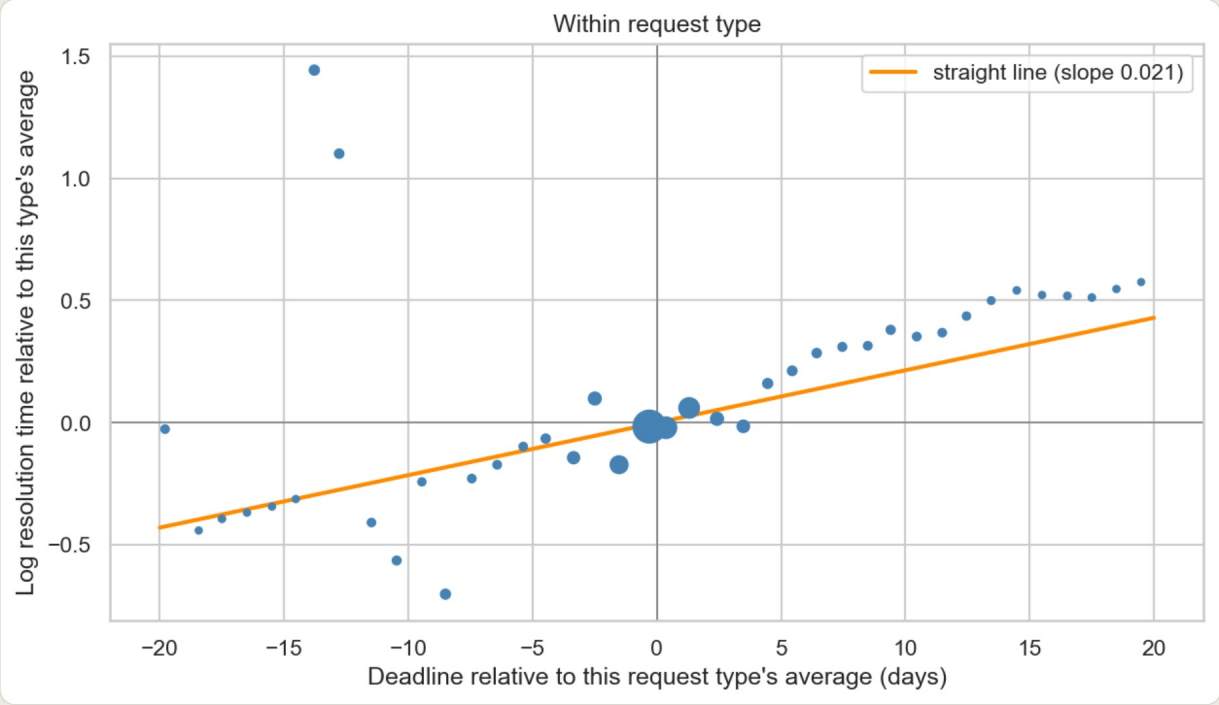


Each point is the average of log(1 + days) for that hour, weekday, or month.

The calendar bends, but the swing is small: 1.5 on this axis is about 3.5 days and 1.7 is about 4.5 days.

MODEL 2 OF 4 · LINEAR REGRESSION

Deadline within request type



Both axes are the difference from the request type's own average; each dot is a bin of many requests.

Once the type is known, an extra deadline day adds little to resolution time.

MODEL 2 OF 4 · RESULT

PRELIMINARY RESULTS

Linear regression results, 2025 test

Model	MAE (days)	RMSE	R^2 (days)	R^2 (log)
One median for every request	14.71	38.88	−0.115	−0.035
Median of each request type	11.92	35.96	0.047	0.264
Linear regression	11.59	35.87	0.051	0.315

2025 test scores for the two median rules and the linear regression.

The regression beats the type median by about 0.3 days of MAE, from 11.9 to 11.6.

METHODS IMPLEMENTED · DESIGN OF THE EXPERIMENT

Two feature sets: what we built, and what we said we built

Set A — implemented

5 columns

Exactly what part 2's linear regression fits.

- sla_days
- request_type
- month
- weekday
- hour

Set B — presented

12 columns

What our methodology slide describes. Set A plus:

- Agency
- MethodReceived
- Neighborhood
- CouncilDistrict
- Latitude
- Longitude
- has_valid_coords

Part 2 dropped these for reasons specific to linear models: agency is **collinear** with request type, and neighbourhood explains little as an **additive shift**. Neither objection binds a tree — so comparing A against B is partly an audit of that gap.

MODEL 3 OF 4 · RANDOM FOREST

METHODS · RESULTS

Random forest — the first model to clearly beat the lookup table

What it is

300 **deep** trees, each grown on a bootstrap sample of the rows and a random subset of columns at every split, then averaged. Individual trees overfit badly; their errors are partly independent, so the average is far steadier.

Categoricals: integer codes. A split can only cut that arbitrary ordering.

Tuned on a 150k subsample: max_features = 0.5, min_samples_leaf = 20

2025 test result

10.49 days MAE · set B

Features	MAE	RMSE	R^2 log
Set A	10.84	34.42	0.332
Set B	10.49	33.72	0.360

Beats the type median by 1.43 days — where the regression managed 0.33. The extra features **help** it.

What it tells us. A model that can use these predictors conditionally — neighbourhood inside one request type, rather than as the same shift everywhere — finds real signal the linear model could not reach.

MODEL 4 OF 4 · GRADIENT BOOSTING METHODS · RESULTS

XGBoost — the best model, but only on the smaller feature set

What it is

Shallow trees fit in sequence, each one on what the ensemble still gets wrong. Predictions are the running sum, so the fit is built up in small corrective steps rather than averaged down from overfit trees.

Categoricals: enable_categorical partitions levels **natively**, grouping them by gradient statistics.

tree_method = hist · max_depth 6 · min_child_weight 20 · 222 rounds by early stopping

2025 test result

10.41 days MAE · set A

Features	MAE	RMSE	R² log
Set A	10.41	33.94	0.358
Set B	10.83	34.60	0.354

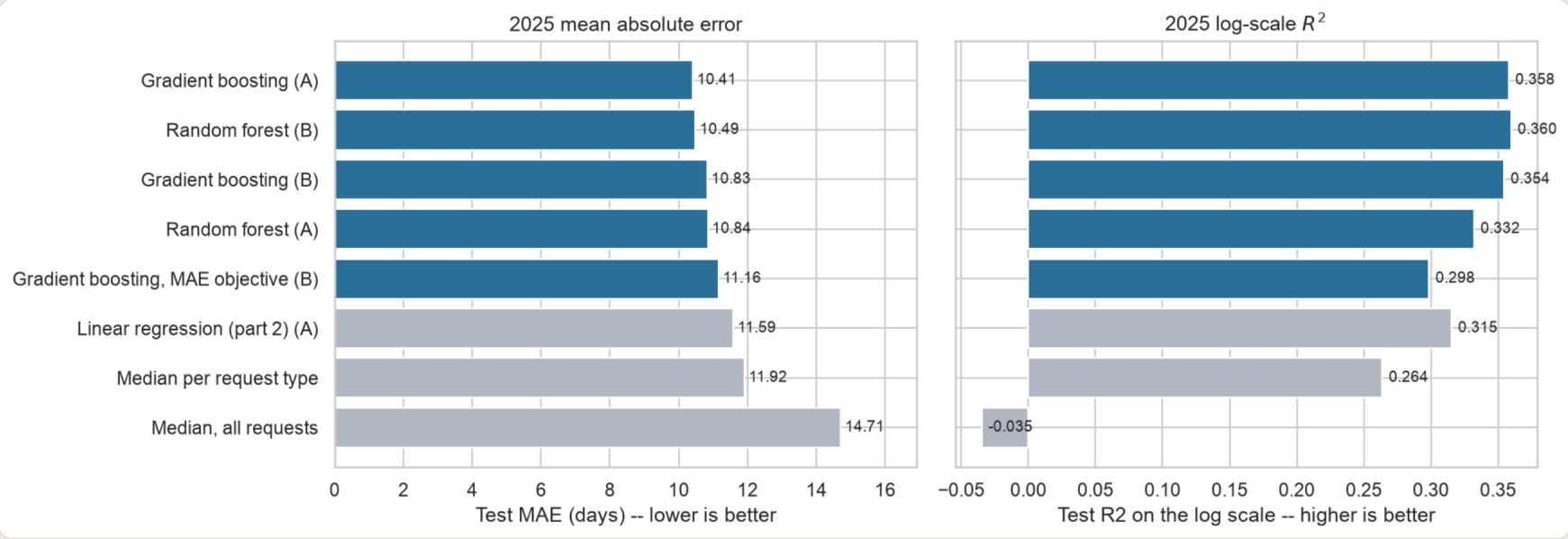
Best overall at 10.41 — but the extra features **hurt** it, the opposite of the forest.

What it tells us. Two tree models, same rows and same split, disagree about whether the **same seven columns** are worth having. That contradiction is the finding we spent the rest of the project explaining.

PRELIMINARY RESULTS · ALL FOUR MODELS

Comparing the four models on the same held-out year

10.41
best MAE · XGBoost



1 · Median baseline
14.71 → **11.92** per type. The bar.

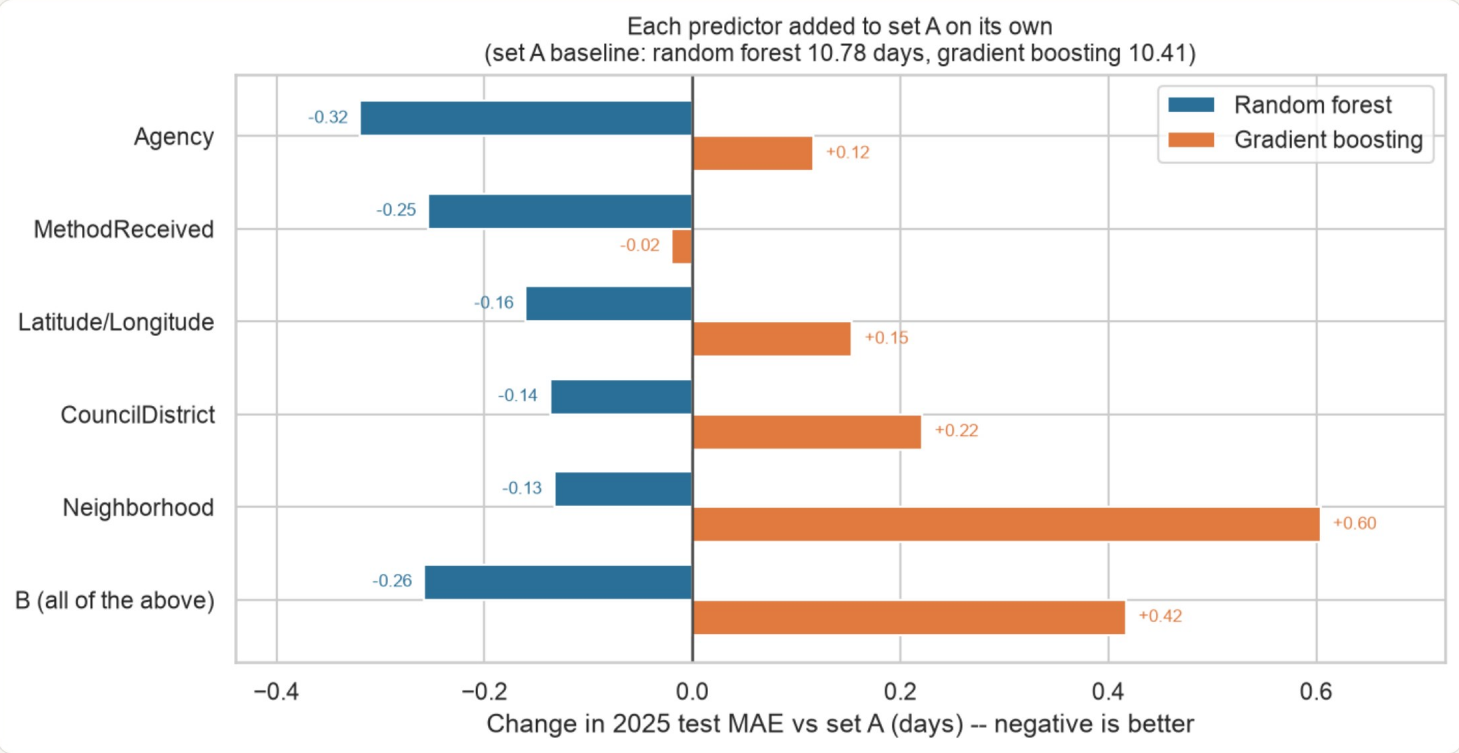
2 · Linear regression
11.59. Clears the bar by 0.33.

3 · Random forest
10.49. Clears it by 1.43.

4 · XGBoost
10.41. Clears it by 1.51.

PRELIMINARY RESULTS · THE CENTRAL FINDING

Almost every feature that helps the forest hurts XGBoost



Forest: all bars left

Every added column improves it. **Agency −0.32** is worth more alone than all seven together.

XGBoost: all bars right

Every column but one makes it worse. **Neighbourhood +0.60** is bigger than our whole gain over part 2.

Below ~0.05 days is noise: column ordering alone moves a forest by 0.04.

One mechanism explains the mirror: how each model reads a categorical

The forest cannot see request type

Integer codes carry no meaning, and a split can only cut that ordering. So the forest **reaches for proxies** — agency, and above all the city deadline.

Permutation importance ranks **sla_days first**, at more than twice request type — even though part 2 showed the deadline is **91% determined** by request type.

XGBoost already has it

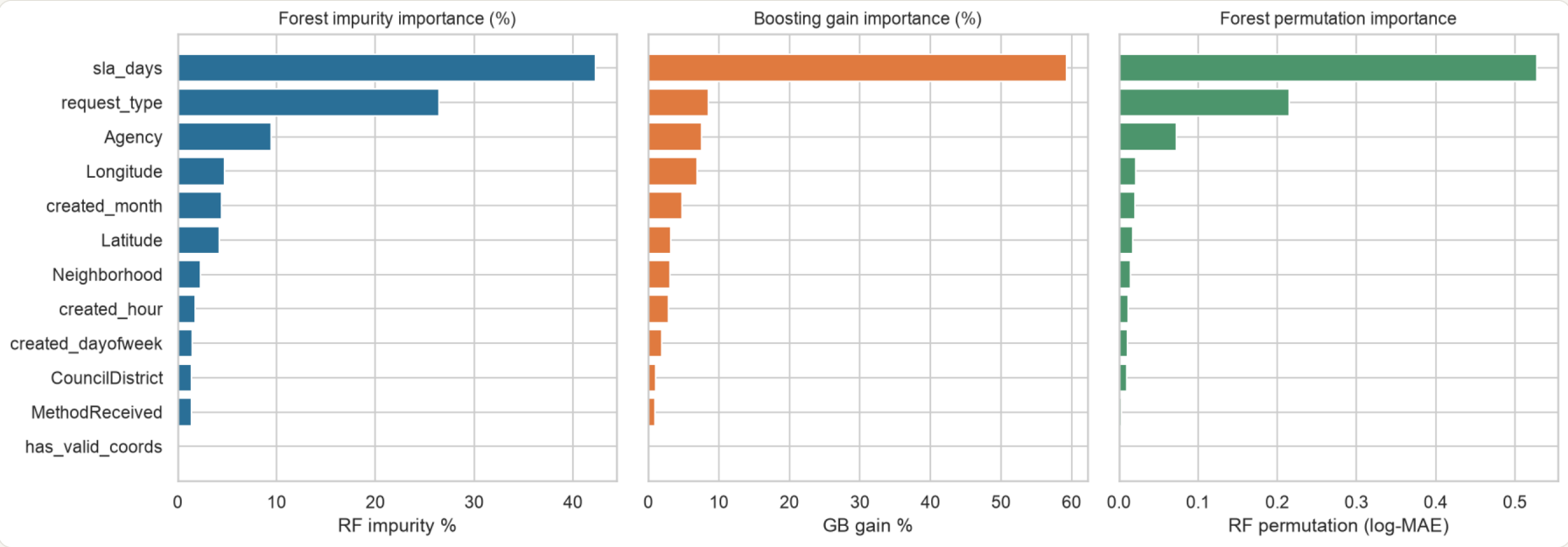
Native partitioning extracts request type directly, so agency is **redundant** — it adds no signal, only surface to overfit.

Given 280 neighbourhoods it carves out fine structure that is **genuinely there in 2024** and **gone in 2025**.

The blunt encoding generalised better than the sharp one — not because it was better, but because it **could not represent what failed to transfer**.

PRELIMINARY RESULTS · INTERPRETATION

Three importance measures, three different answers



Impurity (left) inflates columns with many split points — sla_days has 113,995 distinct values.

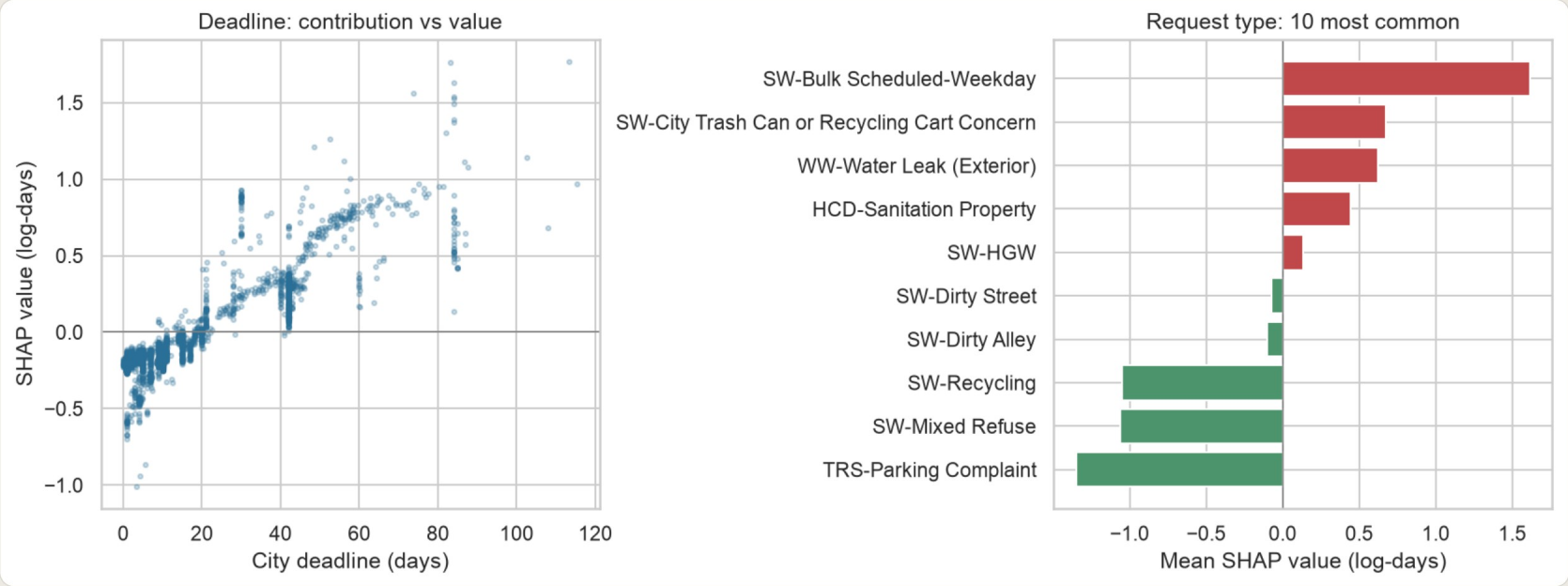
Permutation (right) is held out and cardinality-proof. It **agrees**: the forest really does lean on the deadline.

Neighbourhood is 4th for XGBoost at 7.5% of gain, but 10th for the forest — exactly the column that fails to transfer.

PRELIMINARY RESULTS · INTERPRETATION

SHAP: how each factor moves a single prediction

4.05
days · base prediction

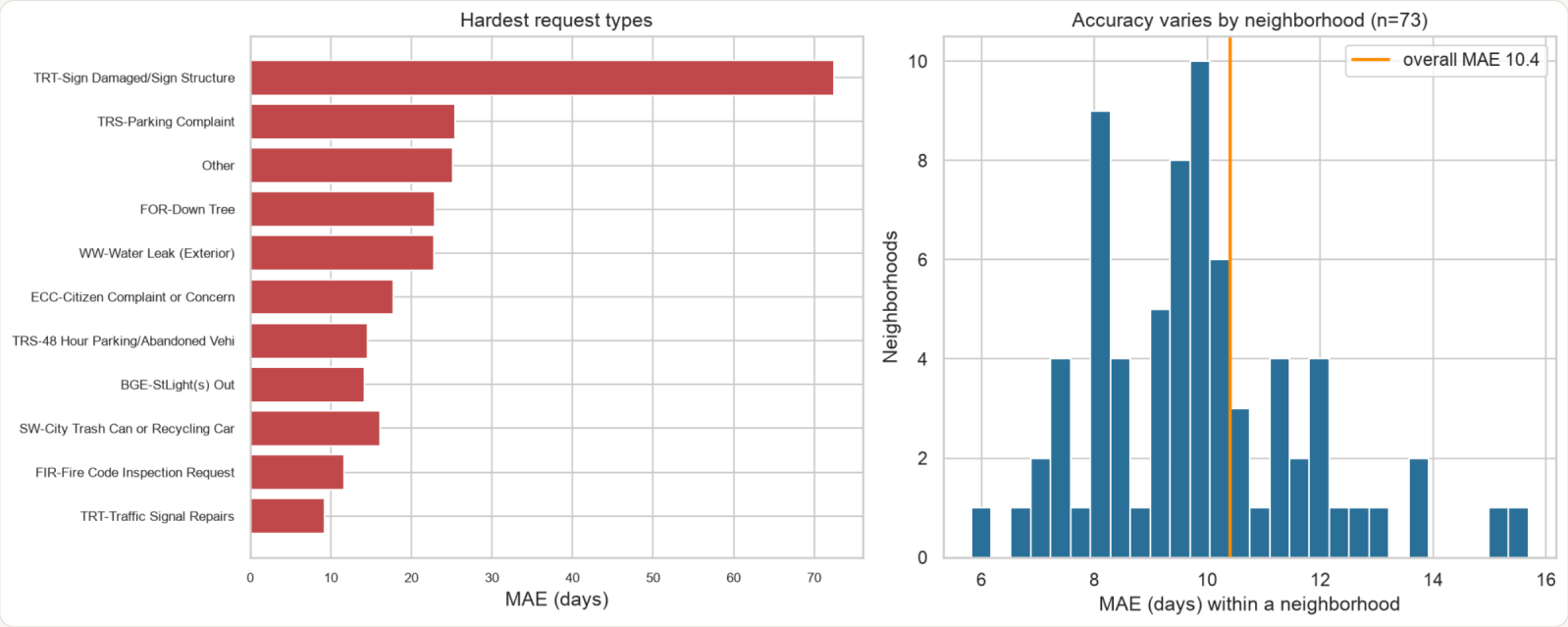


Deadline (left). Contribution rises with the deadline and crosses zero at about **three weeks**: a short deadline pushes a prediction down, a long one up.

Type (right). Parking complaints are pushed **down 1.35** log-days — correct for the typical case, whose median is 0.46 days, and catastrophic for the tail.

PROBLEMS ENCOUNTERED · ERROR ANALYSIS

“10.4 days” averages over wildly unequal cases



72 days
MAE on damaged signs, bias +69

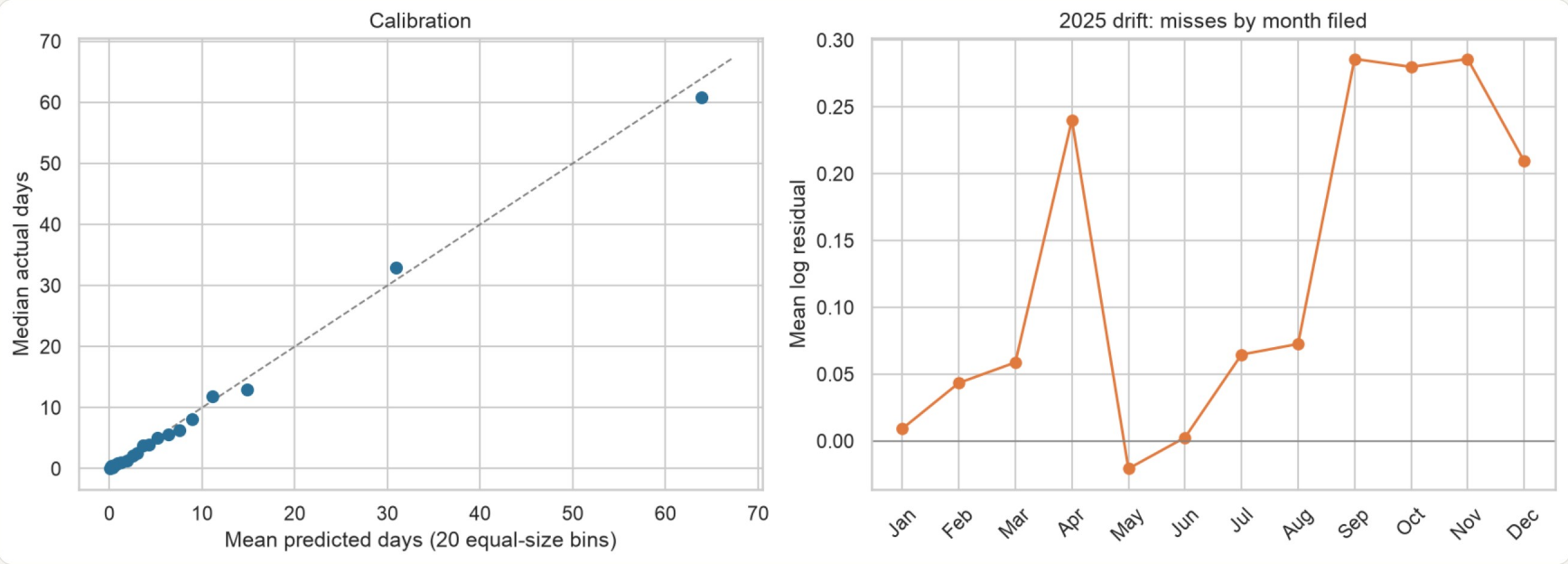
0.01 days
MAE on pothole pickup — it closes on creation

5.8 – 15.7
days MAE across 73 neighbourhoods

73 of 73
neighbourhoods where we are optimistic

PROBLEMS ENCOUNTERED · THE OBSTACLE WE HAVE NOT SOLVED

Every model is optimistic in late 2025



Calibration holds (left). Across its own prediction range the model tracks the diagonal — it is not broken, it is **systematically late**.

Drift (right). Mean log residual is ~0 through June, then **+0.28 to +0.29** for September–November. Requests filed then took longer than a 2024-trained model expects.

No model class fixes this. Part 2 found it too. It is the city changing, not a shortcoming of the fit.

UNSUCCESSFUL APPROACHES, AND WHAT WE LEARNED

Our validation design is broken — and it caught us three times

Fixing XGBoost's neighbourhood overfit — ranked backwards

Categorical setting	Validation	Test 2025
default	8.221	10.826 worst
threshold 16, λ 10	8.261 worst	10.467 best
threshold 8, λ 50	8.199 best	10.608

Restricting the splits **does** fix it, 10.83 → 10.47. Our validation quarter ranked that winner **last**.

Backwards in sign

Validation said set B beat set A by 0.17 days. The test year said A beat B by **0.42**.

Optimising MAE directly

reg:absoluteerror scored **11.16** against 10.83 for the same features — worse, not better.

Why it happens

A quarter inside 2024 measures generalisation across **months**. The test set measures it across **years**.

CHANGES TO THE ORIGINAL PLAN

Four deliberate departures from the proposal

1

Scope widened to all agencies

The proposal covered environmental and housing requests only. We model every agency except 311 call-centre and city-generated work — a larger, more representative sample, and a change we should have flagged sooner.

2

The city's deadline became a feature

DueDate – CreatedDate is known at filing and is the forest's most-used predictor. It was not in the proposal.

3

We added a validation split — and learned it was the wrong one

Oct–Dec 2024 was carved out so hyperparameters never touch the test year. Correct in principle; the previous slide shows why it still misleads.

4

Features audited column by column

Rather than fixing one feature set, we ran both and added each column separately. That is the only reason we found the mirror effect.

PLAN FOR THE FINAL PROJECT

Six steps, in priority order

FIRST · BLOCKS EVERYTHING ELSE

1 · Replace the validation design

Rolling origin: tune on 2024, validate on early 2025. Every comparison in this talk depends on getting this right.

2 · Attack the drift

Features describing conditions at filing: agency backlog, recent median for that type, days since series start.

3 · Settle administrative closures

Decide whether requests that close on creation belong in the target at all.

ANSWERS THE RESEARCH QUESTION

4 · Decompose neighbourhood error within request type

Turns a measurement into the equity finding we actually asked for.

5 · Join Census ACS data

Replaces 280 opaque levels with interpretable quantities — and may generalise where raw neighbourhood identity did not.

6 · Model the tail

Quantile regression, or predicting whether a request will miss its deadline at all.

WHERE WE ARE

Three things to take away

10.41

days of error, down from 11.59

XGBoost beats the per-type median lookup by 1.51 days, where part 2's regression managed 0.33. The tree models earned their place.

±0.6

days, same column, opposite sign

"Which features help" has no answer independent of the model. How each one reads a categorical decides it.

3×

our validation misled us

Validating inside the training year measures the wrong kind of generalisation. Fixing it is step one.

Questions — and we would value your view on scope and on the administrative closures.

Team Chingu · Mani & Yeo Eun